Interrogation Avancée en SQL : Jointures Relationnelles, Regroupements (GROUP BY & HAVING) et Ordre d'Éléments

Thème principal : Bases de Données Relationnelles et Langage SQL Avancé

Synthèse exécutive : Ce cours fondamental de la ressource R1.05 détaille les mécanismes d'interrogation multi-tables et d'analyse statistique dans un Système de Gestion de Base de Données Relationnel (SGBDR). L'étudiant y apprend à réassembler des données normalisées grâce aux jointures (INNER JOIN, LEFT JOIN) tout en évitant le piège critique du produit cartésien, ainsi qu'à structurer des calculs synthétiques par groupes à l'aide des clauses GROUP BY et HAVING. Le fil conducteur pédagogique repose sur la compréhension approfondie du cycle séquentiel d'exécution des requêtes SQL par le moteur de base de données (FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY), clé essentielle pour concevoir des requêtes performantes et réussir les évaluations de partiel.

1. Les Jointures Relationnelles : Interrogation Multi-Tables (INNER JOIN vs LEFT JOIN)

Dans une base de données relationnelle normalisée, les informations ne sont jamais regroupées dans un tableau plat unique afin d'éviter la redondance d'information et les anomalies de mise à jour. La jointure constitue l'opération algébrique centrale permettant de reconstruire la vue logique des données au moment précis de l'interrogation. Lorsqu'une requête s'exécute, l'évaluation débute dans la clause FROM. Pour croiser deux tables (par exemple Etudiant et Note), il est impératif de déclarer explicitement le critère d'équivalence reliant la clé étrangère de la table fille à la clé primaire de la table mère (ex: ON E.id_etud = N.id_etud). Omis ou mal formulé, le SGBD effectue un produit cartésien combinant systématiquement chaque enregistrement de la première table avec toutes les lignes de la seconde. Pour une table de 1 000 étudiants et 50 000 notes, un produit cartésien générerait 50 millions de lignes intermédiaires en mémoire, entraînant un ralentissement extrême ou un crash du serveur de base de données. La distinction entre jointure interne et externe repose sur la gestion des données absentes : l'INNER JOIN élimine toute ligne qui ne possède pas de pendant exact dans la seconde table, alors que le LEFT JOIN garantit la présence de tous les enregistrements de la table positionnée à gauche du mot-clé JOIN, en injectant des valeurs NULL pour combler l'absence de données à droite.

- • Normalisation & Équijointure : Séparation des entités métiers dans des tables distinctes pour éliminer la redondance, puis réassociation explicite via Clé Primaire (PK) et Clé Étrangère (FK).
- • INNER JOIN (Jointure Interne) : Opération d'intersection ne restituant que les combinaisons de n-uplets pour lesquelles le prédicat d'association défini dans la clause ON est strictement vérifié dans les deux tables.
- • LEFT JOIN (Jointure Externe Gauche) : Opération conservant l'intégralité des enregistrements de la table de gauche, en complétant par la valeur NULL les attributs de la table de droite lorsqu'aucune correspondance n'est trouvée.
- • Produit Cartésien (CROSS JOIN) : Évaluation chaotique résultant de l'omission du critère d'association (clause ON), croisant chaque ligne de la table A avec l'intégralité des lignes de la table B (N x M lignes).

Cas concret enseignant : *Le professeur illustre la différence d'application à travers deux cas du quotidien de l'IUT de Bayonne et du Pays Basque : 1. INNER JOIN : Comparable aux portiques d'accès sécurisé du campus d'Anglet. Seuls les étudiants possédant une carte valide ET une inscription effective dans le cours peuvent franchir la porte. Si l'une des deux conditions manque, l'accès est refusé et l'étudiant n'apparaît pas dans la salle. 2. LEFT JOIN : Comparable à la liste d'appel globale éditée par le secrétariat. Tous les étudiants inscrits sur le registre de l'IUT y figurent obligatoirement. Si un étudiant n'a rendu aucun devoir dans une matière, son nom est conservé sur la feuille d'appel et la case de note affiche la mention 'NULL'.*

2. Regroupement de Données et Agrégation Statistiques : Clauses GROUP BY et HAVING

L'analyse décisionnelle et la production de bilans imposent fréquemment de calculer des métriques globales (moyenne générale, somme des ventes, nombre d'inscrits par département). Le langage SQL fournit la clause GROUP BY pour opérer un partitionnement de la table initiale en plusieurs sous-groupes disjoints. Dès que la clause GROUP BY est activée, une règle sémantique stricte s'impose : le moteur SQL ne traite plus des lignes individuelles mais des paquets de lignes. Par conséquent, il devient formellement interdit de projeter dans la clause SELECT une colonne qui varie d'une ligne à l'autre au sein d'un même groupe sans indiquer au SGBD comment la réduire à une seule valeur. C'est pourquoi chaque attribut présent dans le SELECT doit impérativement figurer dans la liste des colonnes du GROUP BY ou être englobé dans une fonction agrégée (COUNT, AVG, SUM, MIN, MAX). Une erreur conceptuelle récurrente lors des évaluations concerne la confusion entre le rôle de la clause WHERE et celui de la clause HAVING. La clause WHERE agit comme un filtre d'entrée sur les tuples individuels AVANT la constitution des groupes. La clause HAVING intervient quant à elle APRÈS le partitionnement et l'évaluation des agrégats pour écarter les groupes ne répondant pas aux critères statistiques demandés.

- • Fonctions d'Agrégation (COUNT, SUM, AVG, MIN, MAX) : Opérations statistiques réduisant un sous-ensemble de tuples à une valeur scalaire synthétique unique.
- • Clause GROUP BY : Subdivise le jeu de données filtré par le WHERE en sous-groupes homogènes partageant des valeurs identiques sur un ou plusieurs attributs.
- • Règle d'or de la projection sous GROUP BY : Toute colonne figurant dans le SELECT doit obligatoirement soit appartenir à la clause GROUP BY, soit être enserrée dans une fonction d'agrégation.
- • Clause HAVING : Prédicat de restriction s'appliquant exclusivement APRÈS la phase de regroupement sur les sous-groupes formés et sur les résultats des fonctions agrégées.

Cas concret enseignant : *L'enseignant illustre le GROUP BY et le HAVING en prenant l'exemple du dépouillement physique des copies d'examen dans l'amphithéâtre de l'IUT : 1. GROUP BY : Le professeur prend la pile globale de toutes les copies rendues et les répartit physiquement dans trois bannettes distinctes selon le département d'origine (Informatique, GEA, TC). 2. Agrégation (AVG) : Pour chaque bannette ainsi constituée, il calcule la note moyenne obtenue par le groupe. 3. HAVING : Si le directeur de l'IUT demande de ne retenir que les départements ayant obtenu une moyenne globale supérieure à 12/20, c'est la clause HAVING qui écarte la bannette non conforme. En revanche, la clause WHERE aurait été utilisée au tout début du processus pour jeter à la poubelle les copies d'étudiants pris en situation de fraude avant même le tri par département.*

3. L'Ordre Logique d'Éxécution d'une Requête SQL par le Moteur SGBD

Pour rédiger du code SQL correct et éviter les erreurs d'exécution mystérieuses, il est vital de comprendre que l'ordre d'écriture d'une requête SQL ne correspond absolument pas à son ordre d'exécution interne par le SGBD. Chronologie logique de traitement par le moteur SQL : 1. FROM & JOIN : Le moteur charge les tables sources et effectue les équijointures en mémoire. 2. WHERE : Il applique le premier filtre ligne par ligne sur la table résultante pour éliminer les tuples non pertinents. 3. GROUP BY : Il partitionne les lignes restantes en sous-groupes sur la base des valeurs des colonnes spécifiées. 4. HAVING : Il filtre les sous-groupes formés en évaluant les conditions portant sur les fonctions d'agrégation. 5. SELECT : Il effectue les calculs d'agrégation définitifs, applique les alias et extrait les colonnes demandées pour la projection finale. 6. ORDER BY : Il prend le jeu de résultats final et procède au tri d'affichage (ASC ou DESC). Cette séquence explique scientifiquement pourquoi la clause WHERE ne peut jamais contenir de fonction d'agrégation : au moment où la clause WHERE s'exécute (étape 2), le partitionnement de l'étape 3 n'a pas encore eu lieu. Le SGBD ne dispose donc d'aucun groupe sur lequel calculer la moindre moyenne ou le moindre décompte.

- • Séquence d'Évaluation SQL : Ordre chronologique dans lequel le processeur de requêtes interprète le code : FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.
- • Impossibilité de l'agrégation dans WHERE : La clause WHERE étant évaluée à l'étape 2, il est physiquement impossible d'y utiliser une fonction d'agrégation (ex: WHERE AVG(note) > 10) car les groupes n'existent pas encore.
- • Alias de colonnes : Les alias attribués dans la clause SELECT (ex: SELECT AVG(note) AS moyenne) ne sont utilisables que dans la clause ORDER BY, car SELECT est interprété après HAVING et WHERE.

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : C'est une impossibilité liée au cycle d'exécution du langage SQL. La clause WHERE est évaluée à l'étape 2 sur la table brute, ligne par ligne, bien AVANT que le SGBD ne procède au regroupement par la clause GROUP BY (étape 3). À l'étape 2, les sous-groupes n'existant pas encore, la fonction d'agrégation AVG(note) n'a aucun sens mathématique pour le moteur SQL. Tout filtre portant sur un calcul d'agrégation doit obligatoirement être placé dans la clause HAVING, qui est évaluée à l'étape 4, juste après le regroupement.

Question étudiant : Que se passe-t-il si j'oublie d'inclure dans la clause GROUP BY une colonne présente dans la clause SELECT ?
Réponse du professeur : Dans un SGBDR respectant la norme SQL (comme PostgreSQL ou Oracle), le moteur de base de données stoppe la requête et renvoie une erreur de syntaxe sémantique de type 'not a GROUP BY expression'. En effet, si vous regroupez les lignes par identifiant d'étudiant et que vous demandez d'afficher la matière dans le SELECT sans l'inclure dans le GROUP BY ni lui appliquer une fonction comme MAX(), le SGBD se retrouve face à plusieurs matières différentes pour un même étudiant et ne sait pas laquelle choisir pour constituer la ligne unique du groupe.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle du secrétariat de l'IUT de Bayonne composée des tables : - Etudiant (id_etud, nom, id_promo) - Note (id_note, id_etud, matiere, note) Écrire la requête SQL complète permettant d'afficher le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants possédant au moins 2 notes enregistrées dans cette matière et obtenant une moyenne stricte supérieure à 12/20. Le résultat final doit être trié de la meilleure moyenne à la moins bonne.

Démarche de résolution :
Étapes méthodologiques détaillées de construction pas à pas : 1. Assemblage des tables (FROM / JOIN) : Les informations 'nom' et 'note' résidant dans des tables séparées, on effectue une équijointure interne dans la clause FROM via la condition 'INNER JOIN Note N ON E.id_etud = N.id_etud'. 2. Filtrage des lignes individuelles (WHERE) : Avant de regrouper, on élimine toutes les notes des autres matières avec le filtre 'WHERE N.matiere = 'R1.05''. 3. Partitionnement en groupes (GROUP BY) : On regroupe les enregistrements par étudiant. Pour se conformer à la règle d'or SQL, on inclut 'E.id_etud' (clé primaire garantissant l'unicité) et 'E.nom' (attribut projeté) dans le 'GROUP BY E.id_etud, E.nom'. 4. Filtrage des groupes agrégés (HAVING) : On formule les deux contraintes statistiques requises : au moins 2 notes ('COUNT(N.id_note) >= 2') ET une moyenne > 12 ('AVG(N.note) > 12.0') dans le HAVING. 5. Projection et calculs (SELECT) : On sélectionne le nom de l'étudiant et la fonction d'agrégation renommée via 'SELECT E.nom, AVG(N.note) AS moyenne'. 6. Tri du jeu de résultats (ORDER BY) : On applique le tri décroissant sur l'alias du calcul d'agrégat via 'ORDER BY moyenne DESC'.

```
SELECT E.nom, AVG(N.note) AS moyenne
FROM Etudiant E
INNER JOIN Note N ON E.id_etud = N.id_etud
WHERE N.matiere = 'R1.05'
GROUP BY E.id_etud, E.nom
HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12.0
ORDER BY moyenne DESC;
```

Résultat : La requête renvoie une vue tabulaire à deux colonnes ('nom', 'moyenne') listant exclusivement les étudiants qualifiés en R1.05 (ayant au moins 2 notes et plus de 12/20 de moyenne), ordonnés par performance académique décroissante.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque principal encouru en cas d'omission de la clause 'ON' dans une jointure SQL entre deux tables comportant respectivement 500 et 2 000 lignes ?

R : L'omission de la clause ON génère un produit cartésien (CROSS JOIN). Le SGBD va croiser chaque ligne de la première table avec toutes les lignes de la seconde, produisant un jeu de données intermédiaire de 1 000 000 de lignes (500 x 2 000), ce qui provoque un surcoût mémoire massif et un risque de plantage.

Q : Dans quel ordre précis le SGBD évalue-t-il les clauses d'une requête SQL comportant FROM, WHERE, SELECT, GROUP BY, HAVING et ORDER BY ?

R : L'ordre logique exact d'exécution par le moteur SQL est : 1. FROM (et JOINs), 2. WHERE, 3. GROUP BY, 4. HAVING, 5. SELECT, 6. ORDER BY.

Q : Quelle est la différence exacte de résultat entre un INNER JOIN et un LEFT JOIN lors du croisement d'une table 'Etudiant' et d'une table 'Note' ?

R : L'INNER JOIN ne conserve que les étudiants ayant au moins une note enregistrée dans la table Note. Le LEFT JOIN conserve la totalité des étudiants de la table Etudiant : pour ceux qui n'ont aucune note enregistrée, la ligne est conservée et les champs de la table Note prennent la valeur NULL.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765